# Edge Detection

Week 5 — Session 2: Canny, thresholds & clean contours

Practical Image Processing

Lecture 5.2 ▪ 60–70 min

# Recap of Session 1

- An edge = a large **gradient**; we built **Sobel**/**Prewitt**/**Scharr**.
- Magnitude $\sqrt{g_x^2 + g_y^2}$ (strength) and direction $\text{atan2}(g_y, g_x)$.
- **Laplacian**/**LoG** for zero-crossings; always **smooth before differentiating**.

**Today (Session 2)**

The thresholded Sobel magnitude is thick and broken. We turn it into thin, connected contours with the **Canny** detector, then feed DocScan.

# Roadmap — Session 2

Why a raw gradient isn't enough

Step by step

Canny in practice

In-class exercise

Deep dive

DocScan & wrap-up

# Why a raw gradient isn't enough

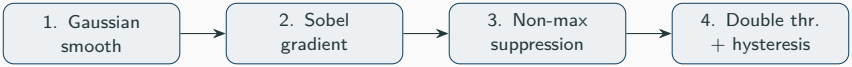
# The problem with thresholded magnitude

- Threshold too low $\Rightarrow$ noise everywhere; too high $\Rightarrow$ real edges break up.
- Edges come out **several pixels thick** — the gradient ramps up and down across the edge.
- Edges are **broken**: intensity dips below threshold along a real contour.

**✕ Pitfall**

One global threshold can never be both sensitive (catch faint edges) and clean (reject noise) at the same time. We need something smarter.

# Canny: the gold-standard pipeline




1. **Smooth** with a Gaussian (kill noise).
2. **Gradient** magnitude & direction (Sobel).
3. **Non-maximum suppression**: thin thick edges to 1-pixel ridges.
4. **Double threshold + hysteresis**: keep strong edges, and weak ones *connected* to strong.

**Key idea**

Canny is not a kernel — it's a 4-step algorithm producing clean, thin, connected edges.

# Step by step

# Steps 1–2: smooth, then gradient

- **Step 1 — smooth:** a 5×5 Gaussian removes the noise that would fake edges (Session 1's lesson made concrete).
- **Step 2 — gradient:** Sobel gives magnitude $M$ and direction $\theta$ at every pixel.
- $\theta$ is snapped to one of four directions: $0°, 45°, 90°, 135°$ — the neighbours we will compare against in step 3.

**Note**

Steps 1–2 are exactly what you built last session. Canny's novelty is steps 3–4.

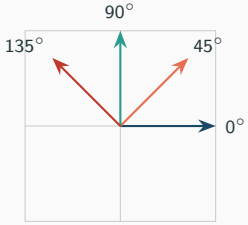
# Step 3: non-maximum suppression (NMS)

- A real edge is a **ridge** of the magnitude. We keep only the ridge *crest*.
- For each pixel, look at the two neighbours *along the gradient direction*.
- Keep the pixel only if its magnitude is $\geq$ both. Otherwise set it to 0.
- Result: the multi-pixel-thick edge collapses to a **1-pixel** line.




$80 > 20$ and $80 > 35$: **keep**. If either neighbour were larger, this pixel is suppressed.

# The four gradient directions for NMS




- The continuous angle $\theta$ is **snapped** to the nearest of $0/45/90/135^\circ$.
- That fixes which two neighbours to compare against in NMS (the pixels straddling the ridge).
- Example: $\theta = 53^\circ$ (Session 1) snaps to $45^\circ \Rightarrow$ compare the two diagonal neighbours.

**Note**

Snapping to 4 bins keeps NMS a simple neighbour lookup instead of a costly interpolation.

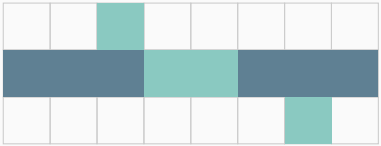
# Step 4: double threshold + hysteresis

- Two thresholds: **high** $T_H$ (definitely an edge) and **low** $T_L$ (maybe).
- $M \geq T_H$: **strong**. $T_L \leq M < T_H$: **weak**. $M < T_L$: **drop**.
- **Hysteresis:** keep every strong edge, plus weak edges *connected* to a strong one; discard isolated weak pixels.

**✔ Good practice**

Hysteresis bridges small gaps along a true edge while rejecting scattered noise responses — it is why Canny gives long, connected contours.

# Hysteresis, visually

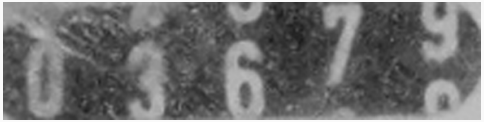


**strong** weak (kept: touches strong) weak (dropped: isolated)

- The weak pixels at $x = 3, 4$ **link** two strong runs $\Rightarrow$ kept.
- The lone weak pixels (top-left, bottom-right) touch no strong edge $\Rightarrow$ dropped.

# Canny in practice

# Canny on real images



meter



Canny edges



plate



# Canny in code

```
blur  = cv2.GaussianBlur(gray, (5, 5), 0)      # step 1: smooth
edges = cv2.Canny(blur, 80, 160)               # low=80, high=160
# OpenCV does steps 2-4 internally: Sobel -> NMS -> hysteresis.
# rule of thumb: high ~= 2-3x low.
```

- `L2gradient=True` uses the exact $\sqrt{g_x^2 + g_y^2}$ (vs the faster $|g_x| + |g_y|$).
- `apertureSize` is the internal Sobel `ksize` (default 3).

# The blur matters: with vs without



Canny *without* pre-blur



Canny *with* 5×5 blur

**✗ Pitfall**

Skipping the blur floods the map with noise edges. The plate shows it too (`plate_canny_noblur` vs `plate_canny`).

# Tuning the two thresholds

- Raise $T_H$: fewer edges, only the strongest survive — contours may fragment.
- Lower $T_L$: more weak edges get rescued by hysteresis — but noise creeps back.
- Keep the ratio $T_H : T_L \approx 2{:}1$ to $3{:}1$; move both together to trade sensitivity for cleanliness.

**Note**

Tune on your *hardest* images (shadowed, low-contrast meter photos), not the easy ones.

# Auto-Canny: thresholds from the median

```python
import numpy as np
def auto_canny(img, sigma=0.33):
    v = np.median(img)
    lo = int(max(0,   (1.0 - sigma) * v))    # ~0.66 * median
    hi = int(min(255, (1.0 + sigma) * v))    # ~1.33 * median
    return cv2.Canny(img, lo, hi)
```

- Fixed thresholds fail across lightings; derive them from the image's own median.
- A robust starting recipe — then fine-tune `sigma` on hard images.

# Closing gaps for the next step

- Even a good Canny contour can have **tiny breaks** where contrast dips.
- Week 9's contour finder wants a *closed* loop around the page — a 1-pixel gap can split it in two.
- Cheap fix: a light **dilation** (or a morphological **close**) fuses the gap without fattening the whole edge.

**✔ Good practice**

Order matters: Canny *first* (thin, clean), then a small close — not a big blur that would have smeared the edge away before detection.

# In-class exercise

# In-class exercise (10 min)

**Trace hysteresis by hand**

A 1D magnitude row (after NMS): `[2, 9, 5, 4, 6, 10, 1]`, with $T_L = 4$, $T_H = 8$.

1. Label each pixel **strong** ($\geq 8$), **weak** (4–7), or **drop** ($< 4$).
2. Apply hysteresis: which weak pixels survive because they connect to a strong one?
3. Which pixels are in the final edge map? What changes if $T_L$ drops to 2?

# Debrief

- Labels: `[drop, strong, weak, weak, weak, strong, drop]` (indices 1 and 5 are strong).
- The weak run at indices 2–4 is bracketed by strong pixels (1 and 5) $\Rightarrow$ all kept by hysteresis.
- Final edge map: indices 1–5 on, 0 and 6 off. A connected 5-pixel contour.
- With $T_L = 2$: index 0 becomes weak but touches nothing strong on its left — still only rescued if a neighbour is strong. Lowering $T_L$ mainly risks pulling in noise.

**Key idea**

Hysteresis links a broken ridge into one contour — the payoff over a plain threshold.

# Deep dive

# Gradient magnitude vs Canny — the difference



Sobel magnitude: thick, noisy



Canny: thin, connected

**✔ Good practice**

For a **contour** (the page boundary), use Canny. For raw **edge strength** (texture/features), use the gradient magnitude.

# Sub-pixel & oriented edges (brief)

- Edge **location** can be refined below one pixel by interpolating the gradient — matters for precise measurement (Week 9).
- The gradient **direction** lets you keep edges of a chosen orientation (e.g. only vertical strokes on the meter).
- Oriented filters (**Gabor**) respond to edges/texture at a chosen angle and scale.

**Math**

Sub-pixel peak: fit a parabola to three magnitudes $M_{-1}, M_0, M_{+1}$ along the gradient; the offset is $\frac{1}{2}\,\frac{M_{-1}-M_{+1}}{M_{-1}-2M_0+M_{+1}}$.

# Edges and deep learning

- A CNN's first layer learns edge/gradient filters *automatically* — often resembling Sobel.
- So the features you compute by hand this week are what a network discovers on its own (Week 13).
- Classical edges stay useful: fast, interpretable, no training data needed.

**Note**

Understanding Sobel/Canny makes the learned first-layer filters legible — you already know what they are.

# Pitfalls & good practice

**✗ Don't skip the blur, don't over-shrink**

No blur ⇒ noise edges. Thresholds too low ⇒ same. Too high ⇒ the page contour breaks into fragments.

**✓ A reliable recipe**

5×5 Gaussian → `auto_canny` → inspect → nudge `sigma`. Then dilate lightly if you need the contour to close for the next step.

# DocScan & wrap-up

# Edges → the DocScan page finder




- Canny gives the page/meter boundary as connected edges.
- Next (Week 9): keep the largest 4-sided contour — the page. Then (Week 11): warp its 4 corners.

**Key idea**

Edges aren't the goal — they're the raw material for finding *shapes* and *structure*.

# What you should now be able to do

- Explain and trace all four Canny steps (smooth, gradient, NMS, hysteresis).
- Run `cv2.Canny`, tune the two thresholds, and implement auto-Canny.
- Say why NMS thins and why hysteresis connects.
- Connect clean edges to the DocScan page-finding pipeline.

# This week's deliverable

**Due before Week 6 (see `project/`)**

1. Compute Sobel-x/y and magnitude on a meter; visualize each (Session 1).
2. Run Canny; show the effect of smoothing (with vs without) and of the two thresholds.
3. Implement auto-Canny from the median; compare to hand-tuned thresholds.
4. DocScan: get clean edges of the page/meter boundary on 3 real photos.

## Smooth, differentiate, thin, connect.

**That's Canny — and that's the edge map DocScan will turn into a page.**